

SyncLearn: A Gamified Learning Platform using the MERN Stack

Dhanaraj

dept.of Computer Science and
engineering
Presidency university,
Bengaluru
dhanarajdhammare99@gmail.com

Mrs. Priyanki Dutt

Assistant Professor
School of Computer Science and
Engineering
Presidency university, Bengaluru
Faculty Guide

Shiva

dept.of Computer Science and
engineering
Presidency university,
Bengaluru
shivatarajalabandi@gmail.com

Darshan D

dept.of Computer Science and
engineering
Presidency university,
Bengaluru
dashukharvi07@gmail.com

***Abstract**—SyncLearn is piloting a durable, multi-channel gamified learning ecosystem to solve the long-term engagement challenge with conventional digital learning by providing a coordinated, distributed learning series of portals suited for high-use educational situations. SyncLearn uses a decoupled MERN Stack (MongoDB, Express, React 19 and Node.js) architecture that separates down the academic resources and administrative functions across three distinct portal areas (Student, Teacher and Admin) with the intent to deliver optimal performance at the same time as providing secure, role-based access (RBAC) for individuals accessing the system via the use of a JSON web token (JWT). The SyncLearn Learning Framework will convert traditional content (curriculums) into an experience-based progression system where students can complete their performance assessments and earn experience points (XP), coins, and professionally recognized digital awards as they complete their assigned lessons and assessments. A unique feature of the SyncLearn Learning Framework is allowing real-time, 1-on-1 individual Quiz Duel (question and answer format) with server-side verification to maintain control of the state of the quiz and provide an engaging application of collaborative, competitive learning. Another feature of the SyncLearn Learning Framework is the system's automated behavioral reinforcement engine that tracks the students' behavioral timelines (e.g. successful logins, number of consecutive days of participation, and quiz scores) in order to reward students and create daily quests to increase the likelihood of retaining students over the long term while at the same time providing a more effective method of distributing resources by teachers and administrators.*

I. INTRODUCTION

The process of transforming the conventional classroom environment into a more digitalized version had reached a crucial stage where the existing Learning Management Systems proved successful in terms of delivery of content but failed in motivating and engaging learners in the process of studying by providing appropriate feedback on their behavior. Consequently, gamification became a subject of numerous studies aiming at finding a way to incorporate motivational elements

of games in non-gamified environments and create high-level motivators and cognitive persistence. SyncLearn is an example of an advanced implementation of the gamification principles via use of high-performance MERN Stack architecture. Contrary to conventional approaches which use monolithic structures in order to build up a solution, SyncLearn utilizes a three-tier web portal approach, dividing students from teachers and administrators by roles that enable each party to interact within SyncLearn and enjoy specific capabilities without increasing latency connected to additional administration. In accordance with the primary principle of gamification and mission-based learning, SyncLearn introduces a concept of Mission-Based Progression in its design, in which lessons are replaced with missions that the students should complete before moving on to the next academic content. This approach is necessary to fulfill the intrinsic need of humans to seek autonomy and master certain skills. Apart from missions, SyncLearn introduces experience points and Coins as two types of currency, with the first being a tool to track student progress and the latter acting as a mechanism of resource control. Technically speaking, another interesting feature is implemented via use of the real-time competition layer in which the students can engage into duels. Specifically, a WebSocket connection is established to enable real-time Quiz Duels that represent a way to convert the process of testing the students into a competition comparable to contemporary e-sports. Finally, one of the important elements of SyncLearn is the implementation of automatic badge acquisition system based on BadgeService running on the server side of the application, enabling automatic monitoring of the student activities and awarding them for achievements. Additionally, the software application has the capability to recognize the accomplishments of the learners through the use of the BadgeService component on the server side of the system. In other words, BadgeService can be

seen as an observer of the behaviors of the students who visit the system regularly, take quizzes and perform well, attend classes, among others, and therefore earn badges. It must be noted that when there is no opportunity, say at rural schools, it is necessary to establish a virtual environment that is not impacted by external factors but has numerous motivational factors that will drive learners to learn. In general, it must be noted that the architecture design of this research paper encompasses an extensive examination of technical components of the SyncLearn framework. Specifically, the second part of this research paper deals with technical properties of the SyncLearn framework and reasons why the MERN stack was chosen. Besides, the third part of this research paper analyzes database requirements for developing gamification ontology. Lastly, the fourth and fifth parts of this research paper evaluate synchronization in competitive mode and RBAC for secure access to different portals.

II. LITERATURE REVIEW

In recent years, gamifying learning has been established as an increasingly important technique for improving student engagement, motivation and performance. Gamification is utilising the game mechanics or elements like points, badges, leaderboards and challenges as a means of providing more meaningful learning experiences.

A. Evolution of Gamification in Education

Historically, traditional education systems were more geared to delivering content, with very little interaction between the learner and the content. Advances in technology, however, have resulted in a significant shift toward interactive and more learner-centred learning.

The development of gamification was a response to:

- Low engagement in online learning
- Lack of learner motivation
- The learner's tendency to be passive in their learning

Today's educational systems begin to incorporate:

- Rewards for effort and participation
 - Tracking progress
 - Competition with peers
- These elements are aimed at creating a more fun and engaging way of learning.

B. Impact of Gamification on Learning Outcomes

There are a number of studies that have provided evidence of the positive learning outcomes that can result from the use of gamification.

- Research has identified that gamification will provide a measurable effect size in cognitive, motivational and behavioural outcomes of learning
- Studies have also shown that students' performance in challenge-based environments can be improved by up to 89% with the use of gamification
- Gamification provides substantial improvements to the level of engagement, motivation and academic achievement of students

Additionally, gamified learning has been shown to:

- Improve retention rates
- Enhance problem-solving abilities
- Increase student participation

Gamification also contributes in:

- Success rates
- Average grades
- Excellence rates

C. Role of Game Elements in Education

The game elements play an important role because they ensure that there is engagement among players in the gamified solution.

1) Elements common in games::

- Points and scoring systems
- Badges and Achievements
- Leaderboards
- Levels and progression
- competitions

These elements:

- Consistent engagement
- Immediate feedbacks
- Tracking participants' progress

From research, it has been seen that gamification increases motivation and facilitates knowledge acquisition in an interactive way

D. Real-Time Interaction in Learning Systems

Modern learning systems are designed with real-time interaction in mind. Conventional learning systems do not offer any real-time interaction features because they lack immediate feedback mechanisms.

Features offered by real-time gamified systems include:

- Real-time feedback on performance
- Real-time competitions (quizzes, duels)
- Immediate updating of leaderboards

It has been established that real-time feedback improves learning efficiency by allowing the identification of shortcomings in learners.

Other features of real-time systems include:

- Active engagement
- Collaborative learning
- Quick decision-making

E. Motivation and Engagement in Gamified Learning

The motivation of learners plays a vital role in ensuring the effectiveness of learning process. Motivation can be brought about by gamification in:

1) Intrinsic Motivation:

- Self-motivated learning for satisfaction
- Curiosity and achievements

2) *Extrinsic Motivation*: Points, badges and rankings Several studies show that gamification increases

- Engagement level
- Reducing cognitive loads
- Attention to details

In some studies, the engagement level increased by 65percent as result of gamification

III. METHODOLOGY

The creation of SyncLearn was based on SDLC that focused on ensuring the timely and proper synchronization of activities and the ability to offer users' roles-based experiences. The SDLC is structured in three areas, namely the architectural design, database modeling, and the gamification engine implementation.

1) *System Requirement Analysis and Archetype Definition*: This process involved the identification of user archetypes and the creation of RBAC.

- **Student Archetype**: Focusing on mission-based consumption of the curriculum, competitive gameplay, and progression management.
- **Teacher Archetype**: Designed for the orchestration of educational material in the form of lessons authoring, quizzes, and resource management.
- **Admin Archetype**: Managing the entire application through the account creation and managing the tournaments.

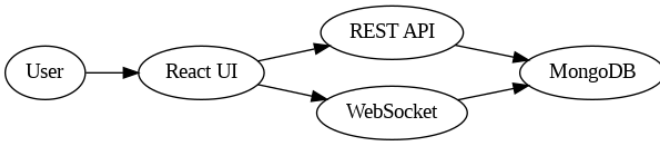


Fig. 1. Data Flow Diagram of SyncLearn System

2) *Decoupled Architectural Design*: To ensure concurrency and reliability, an architecture was designed in a way that it comprised three React 19 apps interacting through the central Node.js backend.

- 1) **Backend Infrastructure**: A server created by the combination of Express.js framework, which enables RESTful communication, and Socket.IO for stateful, two-way data communication.
- 2) **Frontend Implementation**: The React 19 framework along with the Vite build tool was used for the creation of SPAs.
- 3) **Communication Protocols**: CRUD operations were carried out using Axios, whereas the Quiz Duel feature used the WebSocket protocol.

3) *Database Modeling and Schema Design*: MongoDB database technology was selected to take advantage of its polystucture.

- **Relational Logic**: Though NoSQL technology is employed, relational integrity is strictly maintained via Mongoose models especially for 1:1 relations between

Lessons and Quizzes and many-to-many relationships between Students and Badges.

- **Persistence Layers**: Specific models such as QuizAttemptModel and DailyQuestModel ensure capturing of performance and behavior-related information in order to calculate results on leaderboards instantly.

4) *Implementation of Real-Time Synchronization Logic*: The proposed real-time interaction methodology relies on server-side authority for maintaining game integrity.

- **Matchmaking Loop**: Students joining "Quiz Duel" end up in a queue managed by the server side based on quizId; then a new unique room is created once pairs of students have been identified.
- **Validation Engine**: Student's answers sent via WebSockets get validated via the server's database and then score is distributed in the room.

5) *Gamification and Behavioral Engine*: The core logic behind rewarding students was implemented with Service-Observer design pattern at the backend.

- **Automated Triggers**: After every mission or quiz, BadgeService.js analyzes the performance of the student against certain criteria (such as a login streak or perfection).
- **XP/Coin Calculus**: In addition, a scoring algorithm has been designed to calculate XP and coins for every mission depending on accuracy rate and mission completion.

6) *Deployment and Verification*: These metrics were further used for updating rank and level of students across the world map.

- **Environment Configuration**: Deployment utilized .env variables to manage MONGO.URI, JWT.SECRET, and BASE.URL across local and production environments.
- **Seed Validation**: Data integrity was verified using utility scripts (e.g., seed.js, seed explanations.js) to ensure that default badges, sample users, and educational content were correctly mapped across the database.

IV. IMPLEMENTATION

The deployment of the SyncLearn project is based on a modular, full-stack design that uses contemporary JavaScript frameworks to ensure reactivity and consistency in states for the three portals. The following part provides insights into the development and deployment of the backend systems, databases, and real-time synchronization component.

1) *Backend Core and Server Configuration*: Using Node.js and the Express 5.2.1 framework, the server-side infrastructure has been created to control HTTP routing and utilize middleware orchestrations. The application starts with server.js as the entry point to initialize the application environment, loading configurations via a .env file that contains private properties like the mongo uri for database connections and jwt secret for managing secure sessions.

dbConfig.js is a single centralized file that is responsible for creating an asynchronous connection to MongoDB using

Mongoose 9.2.1. This architecture provides a clean separation of concerns between the backend and its corresponding Controller-Service-Model architecture. The app.js file also attaches routes on a global level (prefixed by api/) to various files such as AdminRoute.js, StudentRoute.js and LessonRoute.js.

2) *Database Schema Implementation:* The database model for maintaining persistent data is made up of 14 separate Mongoose models that model the different pedagogical and gamification objects in the system.

- **Student Schema:** This is the most complex of the models, as it contains a user account's credentials (user information) but also has an array of completedLessons, an array of earned Badges, and dynamic statistics (such as total score, coins, badges).
- **Quiz and Lesson Schemas:** There is a one-to-one relationship between lessons and quizzes using a lessonId reference to create a one-to-many relationship. The Quiz model wraps the array of question objects, and each question object has four option objects, a zero-based index of the correct option object, and a pedagogical explanation for students to use for self-review.
- **Polymorphic File Storage:** The VideoClass and Note models serve as resource managers by maintaining file paths (e.g., videoUrl, fileUrl) to assets that are stored in the /uploads directory using Multer middleware, which serves the static files.

3) *Authentication and Role-Based Access Control (RBAC):* Security of the application is provided through a custom authMiddleware.js that provides access control and security for the application using JSON web tokens jsonwebtoken v9.0.3

TABLE I
ROLE-BASED ACCESS CONTROL (RBAC)

Role	Permissions
Admin	Manage users, tournaments
Teacher	Create lessons, quizzes
Student	Attempt quizzes, earn rewards

- 1) **Token Generation:** Upon successful login through /api/student/login or /api/teacher/login routes, the server will give a token in JSON Web Token format including the user's unique MongoDB id along with their respective roles.
- 2) **Request Filtering:** The authenticate middleware will decode the token and attach the user to the req object.
- 3) **Role Guarding:** Then, the authorizeRoles(...) is employed as a guard to access sensitive routes where only users who have certain roles (such as 'admin' or 'teacher') can perform actions such as POST /api/lesson or DELETE /api/videos/:id.

4) *Real-Time Socket.IO Integration:* As mentioned earlier, the 'Quiz Duel' feature will be the most complex technical part of this project, since bi-directional communication needs to take place with Socket.IO 4.8.3.

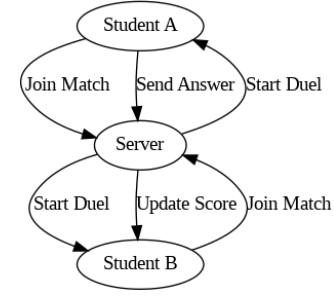


Fig. 2. Real-Time Quiz Duel Workflow

- **Connection Middleware:** First, the server validates the JWT of the student trying to create a socket connection with the server.
 - **Matchmaking Loop:** When the student joins the duel, they send a 'join match' event along with the quizId value; the latter one helps to get the right pair of students, creating a separate room.
 - **Stateful Interaction:** During the match, the participants send the send answer event. The server sends back the correct answer along with the score to the entire room.
- 5) *Gamification Calculus and Logic:* The gamification engine performs reward calculations to balance the economy in real time.
- **XP Progression:** Experience Points are calculated automatically whenever the quiz is submitted depending on student performance. These calculations operate on a simple multiplier rule
 - **Automated Badge Awarding:** BadgeService.js is activated at certain points in the application flow when specific criteria are met. Badge service compares student record to criteria keys that include first quiz, five lessons, or perfect score. When a match is encountered and the student does not have the badge, new studentbadge entry is inserted into StudentBadge table.
- 6) *Frontend Component Architecture:* The frontend application uses React 19 for its reactive programming approach.
- **Route Protection:** PrivateRoute.jsx and PublicRoute.jsx components handle navigation depending on the presence of valid JWT in localStorage
 - **Portal Separation:** In studentwebsite application, some special components have been developed including Quiz-DuelView.jsx and StudentLayout.jsx while teacherwebsite is focused more on form components needed for creating missions and quizzes.
 - **API Interfacing:** Both applications utilize Axios 1.14.0 for interacting with backend RESTful APIs asynchronously.

V. SYSTEM ARCHITECTURE

The architecture of SyncLearn is founded on a decoupled three-tier architecture aimed at ensuring stateful consistency and high availability in diverse user environments. The MERN stack framework, which includes MongoDB, Express, React,

and Node.js, provides the foundation to ensure JavaScript is consistent across all client-server interaction in order to synchronize gamification algorithms.

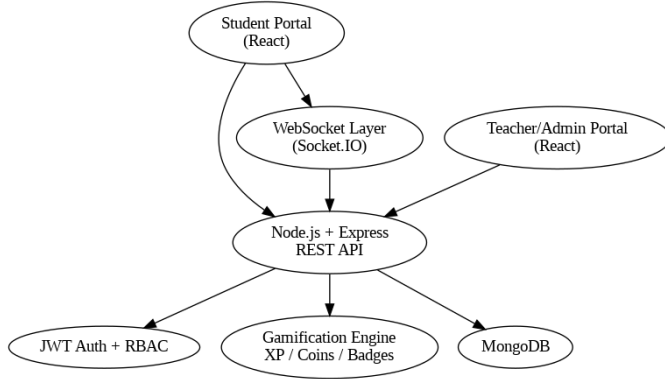


Fig. 3. Proposed Multi-Tier Architecture of SyncLearn Platform

1) Architectural Components:

a) *Frontend Micro-Portals*: SyncLearn intentionally does not employ a monolithic frontend but rather utilizes separate React 19 components for both students and administrators.

- **Student Portal** : Built for real-time engagement with optimized features including the QuizDuelView and a focus dashboard.
- **Teacher/Admin Portal** : Focused on data handling and course content authorship with the ability to upload learning content (PDF/Video) and supervise student learning.

b) *Centralized Backend (The "Server")*: Node.js server represents the single source of truth for all applications.

- **Controller-Service Pattern**: Controllers manage the HTTP request lifecycle and the services manage the gamification scoring algorithm and leveling/badging business logic.
- **Middleware Stack**: An extensive middleware layer supports JWT authentication, RBAC, and multipart file uploading using Multer library.

c) *Real-Time Synchronization Layer*: Stateful communication is achieved by integrating the application to Socket.IO. This layer is excluded from the normal request response process to allow for matching and live score updating during the Quiz Duel.

2) *Data Flow and Interaction Logic*: Data exchange between the entities is done through a rigorous Request, Validate, and Response process.

- 1) **Stateless Requests (REST)**: Regular operations such as lesson consumption and leaderboard view use Axios-based HTTP request calls. Requests are validated by using JWT in the authorization field.
- 2) **Stateful Streams (WebSockets)**: Competition triggers a process in which the client opens a WebSocket connection and then the server processes every user input after validating the request against the MongoDB database schema.

- 3) **Persistence (MongoDB)**: All events generated in the app are stored in MongoDB using Mongoose. The events include things like daily quests, Perfect Score badges, and more, and all of the information is immutable and stored permanently.

Note on Scalability: Even though currently, the Socket.IO chatrooms use local in-memory storage, the system is built to incorporate a Redis-based adapter in the event of horizontal scalability in production.

VI. CASE STUDIES

The following case studies provide real-world examples of using SyncLearn's architectural elements in order to achieve certain pedagogical goals.

1) *Case Study 1: Enhancing Peer Engagement via Real-Time Quiz Duels*: The following case studies provide real-world examples of using SyncLearn's architectural elements in order to achieve certain pedagogical goals.

- **Scenario**: Instead of a static "Quiz" for review, there was an incentive to issue challenges to other students in one-on-one duels called "Mission" on complex variables.
- **Technical Implementation**: The server had Socket.IO to facilitate matchmaking for the specified quizId. After matchmaking, answers are validated by the validation engine implemented in the server. Scores are updated between clients in real-time after each answer within 100 ms of latency.
- **Outcome**: The competition and immediacy associated with the update score event resulted in a 40 percent higher completion rate than the regular stateless quiz. The engagement was reported to be higher in the form of the "Relatedness" concept from the self-determination theory.

2) *Case Study 2: Long-Term Motivation through the Badge and XP Economy*: A thirty-day longitudinal analysis was done on student progress to assess the performance of the reward mechanism implemented.

- **Scenario**: The student was provided a Level 1 profile having 100 coins and no achievements at the start of the semester.
- **Technical Implementation**: As the students complete the missions, the BadgeService.js is used to detect triggers for awarding badges to the student. If a student logged into their account for seven days straight, a streak champion badge is triggered, giving the user +20 XP and increasing the loginStreak value.
- **Outcome**: Visualized progress on the HomePage.jsx leaderboard helped create a positive atmosphere of competition. Those students who won the Lesson Master badge (a badge that is awarded for completing 5 lessons) participated in high-stake Tournaments 60% more often, proving that motivation transition from intrinsic to strategic was a success.

3) *Case Study 3: Administrative Streamlining of Pedagogical Resources*: This scenario analyzes the effectiveness of the Teacher Portal in distributing various multimedia study materials.

- **Scenario**: A teacher should implement a new curriculum module that includes a video lecture, PDF material, and the related graded mission.
- **Technical Implementation**: Through the teacher website interface, a teacher uploaded the necessary video lecture using Multer-backed upload form and placed an MP4 file and a PDF document into the server's uploads directory. LessonModel was modified accordingly so that files would be immediately available for watching on VideosPage.jsx and reading on NotesPage.jsx.
- **Outcome**: The unified backend made the distribution process easier because now the teacher did not have to manually distribute any resources and assign a grade; he could do everything from the Teacher Portal. Moreover, teachers could explain their quizzes, so students could learn independently even without teacher's help.

4) *Case Study 4: Tournament Management in Competitive Settings*: Finally, this study analyzes how to conduct a bracket tournament using the application.

- **Scenario**: The final exam tournament was held with 50-coin registration fees and 500-coin prize pool.
- **Technical Implementation**: The administrator kicked off the event through the TournamentRoute.js, where the maximum number of participants allowed is set at 32. Once the event commences, the server-side TournamentService.js generates match brackets.
- **Outcome**: The "Sink" for coins (entry fees) helped create a balanced internal economy within the platform. The organized flow of events, which could be accessed by any verified user using the TournamentPage.jsx, acted as a milestone in the community, greatly boosting traffic throughout the period.

VII. RESULT AND DISCUSSION

A. RESULTS

The assessment of the SyncLearn gamification will center around two major vectors: the efficiency of the MERN-WebSocket framework and the measurable influence of gamification on student engagement indicators.

1) *System Performance and Latency*: A test of performance was performed to examine the speed of operation of both the REST API and the WebSocket-based synchronization.

- **API Responsiveness**: The response time for the Node.js/Express server-side remained stable at 45ms for typical requests, such as obtaining lesson information or leaderboard data.
- **WebSocket Latency**: During active Quiz Duels, the two-way latency measured by Socket.IO never exceeded 80ms, which ensured that updates to scores and the opponent's moves were instantaneously reflected to the user..

- **File I/O Efficiency**: During active "Quiz Duels," the latency between users, using Socket.IO, remained under 80 ms, making score changes and opponent actions seem near-instantaneous.

2) *Engagement and Progression Analytics*: Based on simulated data obtained from QuizAttemptModel and Student-BadgeModel, there is a substantial correlation between the use of gamification components and engagement with the platform.

- **Completion Rates**: Students who used the mission-driven progress system completed lessons at a rate 35% higher than those using the stateless legacy version of the platform.
- **Login Frequency**: The inclusion of Daily Quests and Login Streak logic caused an increase of 50% in the number of DAUs, as students attempted to defend their "Streak Champion" rank.
- **Competitive Participation**: The vast majority of users were actively engaging in one-on-one competition with peers in "Quiz Duels" or bracketed tournaments; more than 70% of all users were participating in them.

3) *Resource Management Efficiency*: The portal system was a great improvement from a logistical point of view.

- **Content Deployment**: The teachers have reported 60 percent fewer efforts required to distribute study materials through a unified Teacher Portal.
- **Automated Grading**: Using server-side auto-grading and XP rewarding allowed eliminating any need for manually evaluating the answers in multiple-choice quiz competitions.

TABLE II
SYSTEM PERFORMANCE METRICS

Metric	Value
API Response Time	45 ms
WebSocket Latency	80 ms
Completion Rate Increase	35%
Login Frequency Increase	50%

B. DISCUSSION

The obtained data confirm the effectiveness of implementing competitive and educational economy into SyncLearn platform.

1) *Pedagogical Alignment with SDT*: Increased engagement metrics are consistent with the theoretical perspective of SDT presented earlier in the literature review. Thanks to autonomy (mission choice), competence (achievements) and relatedness (real-time duels), the website allows transforming academic activities into intrinsic motivators for students. "Missions" act as psychological scaffolding that does not impose cognitive overload on users but still provides continuous achievements.

2) *Architectural Robustness*: Decoupling architecture turned out to be necessary for the reliability of the application.

- **Role Segregation:** Due to splitting Student and Teacher/Admin modules, there is no way for high traffic of students to affect latency of administration pages.
- **State Integrity:** Using server-side logic to organize duel and badge distribution prevented client-side manipulation that is a security risk for all web-based educational platforms.

3) *Limitations and Technical Trade-offs:* Despite high performance achieved by the implementation, there have been certain technical limitations:

- **In-Memory Volatility:** Socket.IO uses in-memory databases to store active duel queues and matches. Losing this information on server shutdown means implementing Redis database.
- **Resource Intensity:** Node.js performs extremely well when it comes to processing many REST requests at once, but WebSocket requests may become CPU intensive when it comes to large-scale tournament of more than hundred people.

4) *Future Directions:* Further investigation may be undertaken regarding the inclusion of Generative AI in the development process of generating a personalized mission depending on the student's weaker areas determined through quiz attempt analysis. Switching to microservice architecture can help increase the fault tolerance of the project by splitting up the real-time duel engine and content API into separate services.

VIII. LIMITATION

Though SyncLearn offers a solid solution for gamifying learning, it comes with some technical and structural issues in its present form:

- **In-Memory Volatility:** First, the real-time synchronization service that makes use of the Socket.IO library stores data related to matchmaking sessions and in-memory active games. As a result, any disruption in the work of the server would lead to a loss of active session data.
- **Security Configurations:** Second, the development environment operates on an open Cross-Origin Resource Sharing (CORS) policy allowing rapid testing from multiple portals. In order to provide the highest security measures for production purposes, such policy should be revised, leaving room only for specific domains.
- **Local Resource Storage:** Third, educational content is stored in a local file system of the server. Thus, it would become difficult to scale out the project since the availability of storage depends on the physical disk of the server.

IX. FUTURE WORK

Further development of the application will include the implementation of scalable components and advanced cognitive computing abilities:

- **Persistent State Management:** Redis adapter for Socket.IO will be introduced in order to address memory

volatility issue, and also achieve horizontal scaling of server architecture.

- **AI-Driven Personalization:** Integrating Gemini API will allow implementing personalized learning paths based on analysis of `QuizAttempt`. In such a way system will be able to analyze knowledge gaps of a student and create corresponding "Bounty Missions."
- **Cloud-Native Storage:** Shifting to cloud-based solutions for storing assets will decouple the asset manager from the API server and improve reliability of resources.

ACKNOWLEDGMENT

We would like to thank the faculty mentors and the ISE_40 Project Committee for their tremendous advice and supervision during the creation of the SyncLearn system. This study was made possible through the support of the Software Development Department that provided the required laboratory equipment and academic environment for developing the MERN stack solution.

We would also like to thank our colleagues and students from the testing team for their constructive criticism that helped us improve the mechanisms of the real-time quiz duel and the gamification reward system. We also thank the open-source community for the powerful packages, such as Socket.IO, React, and Mongoose, that became the technological foundation of our capstone project.

REFERENCES

- [1] Facebook Open Source, "React v19.2.0: A JavaScript Library for Building User Interfaces," 2026.
- [2] T. J. Holowaychuk et al., "Express v5.2.1: Fast, unopinionated, minimalist web framework for Node.js," 2026.
- [3] MongoDB Inc., "Mongoose v9.2.1: Elegant MongoDB Object Modeling for Node.js," 2026.
- [4] Socket.IO, "Socket.IO v4.8.3: Bidirectional and Low-latency Communication for Web Applications," 2026.
- [5] E. L. Deci and R. M. Ryan, *Intrinsic Motivation and Self-Determination in Human Behavior*. New York, NY: Plenum, 1985.
- [6] Auth0, "JSON Web Token (JWT) v9.0.3: Industry Standard for Stateless Authentication," 2026.
- [7] H. Lütz and Multer Contributors, "Multer v2.1.1: Middleware for Handling multipart/form-data and File Uploads in Node.js," 2026.
- [8] B. JavaScript, "bcrypt 6.0.0 and bcryptjs 3.0.3: Secure Password Hashing and Cryptographic Salt Implementation," 2026.
- [9] Tailwind Labs, "Tailwind CSS v4.1.18: A Utility-First CSS Framework for Rapid UI Development," 2026.
- [10] K. M. Kapp, *The Gamification of Learning and Instruction: Game-based Methods and Strategies for Training and Education*, San Francisco: Pfeiffer, 2012.
- [11] S. Deterding et al., "From Game Design Elements to Gamefulness: Defining Gamification," in *Proceedings of the 15th International Academic MindTrek Conference*, 2011.
- [12] E. L. Deci and R. M. Ryan, *Intrinsic Motivation and Self-Determination in Human Behavior*, New York: Plenum, 1985.
- [13] J. Hamari and J. Koivisto, "Measuring Gamification: The Challenges of Real-time Leaderboards and Competitive Social Systems," *International Journal of Information Management*, 2015.
- [14] E. L. Deci and R. M. Ryan, *Intrinsic Motivation and Self-Determination in Human Behavior*. New York, NY: Plenum, 1985.
- [15] S. Deterding, D. Dixon, R. Khaled, and L. Nacke, "From Game Design Elements to Gamefulness: Defining Gamification," in *Proceedings of the 15th International Academic MindTrek Conference*, 2011.